

Changelog

1.16.0 (2024-07-26)

- Feature: Improve PHP 8.4+ support by avoiding implicitly nullable type declarations. (#318 by @clue)

1.15.0 (2023-12-15)

- Feature: Full PHP 8.3 compatibility. (#310 by @clue)
- Fix: Fix cancelling during the 50ms resolution delay when DNS is still pending. (#311 by @clue)

1.14.0 (2023-08-25)

- Feature: Improve Promise v3 support and use template types. (#307 and #309 by @clue)
- Improve test suite and update to collect all garbage cycles. (#308 by @clue)

1.13.0 (2023-06-07)

- Feature: Include timeout logic to avoid dependency on reactphp/promise-timer. (#305 by @clue)
- Feature: Improve errno detection for failed connections without `ext-sockets`. (#304 by @clue)
- Improve test suite, clean up leftover `.sock` files and report failed assertions. (#299, #300, #301 and #306 by @clue)

1.12.0 (2022-08-25)

- Feature: Forward compatibility with react/promise 3. (#214 by @WyriHaximus and @clue)
- Feature: Full support for PHP 8.2 release. (#298 by @WyriHaximus)
- Feature: Avoid unneeded syscall on socket close. (#292 by @clue)
- Feature / Fix: Improve error reporting when custom error handler is used. (#290 by @clue)
- Fix: Fix invalid references in exception stack trace. (#284 by @clue)
- Minor documentation improvements, update to use new reactphp/async package instead of clue/reactphp-block. (#296 by @clue, #285 by @SimonFrings and #295 by @nhedger)

- Improve test suite, update macOS and HHVM environment, fix optional tests for `ENETUNREACH`. (#288, #289 and #297 by @clue)

1.11.0 (2022-01-14)

- Feature: Full support for PHP 8.1 release. (#277 by @clue)
- Feature: Avoid dependency on `ext-filter`. (#279 by @clue)
- Improve test suite to skip FD test when hitting memory limit and skip legacy TLS 1.0 tests if disabled by system. (#278 and #281 by @clue and #283 by @SimonFrings)

1.10.0 (2021-11-29)

- Feature: Support listening on existing file descriptors (FDs) with `SocketServer`. (#269 by @clue)

```
$socket = new React\Socket\SocketServer('php://fd/3');
```

This is particularly useful when using systemd socket activation like this:

```
$ systemd-socket-activate -l 8000 php examples/03-http-server.php php://fd/3
```

- Feature: Improve error messages for failed connection attempts with `errno` and `errstr`. (#265, #266, #267, #270 and #271 by @clue and #268 by @SimonFrings)

All error messages now always include the appropriate `errno` and `errstr` to give more details about the error reason when available. Along with these error details exposed by the underlying system functions, it will also include the appropriate error constant name (such as `ECONNREFUSED`) when available. Accordingly, failed TCP/IP connections will now report the actual underlying error condition instead of a generic “Connection refused” error. Higher-level error messages will now consistently report the connection URI scheme and hostname used in all error messages.

For most common use cases this means that simply reporting the `Exception` message should give the most relevant details for any connection issues:

```
$connector = new React\Socket\Connector();
$connector->connect($uri)->then(function (React\Socket\ConnectionInterface $conn) {
    // ...
}, function (Exception $e) {
    echo 'Error:' . $e->getMessage() . PHP_EOL;
});
```

- Improve test suite, test against PHP 8.1 release. (#274 by @SimonFrings)

1.9.0 (2021-08-03)

- Feature: Add new `SocketServer` and deprecate `Server` to avoid class name collisions. (#263 by @clue)

The new `SocketServer` class has been added with an improved constructor signature as a replacement for the previous `Server` class in order to avoid any ambiguities. The previous name has been deprecated and should not be used anymore. In its most basic form, the deprecated `Server` can now be considered an alias for new `SocketServer`.

```
// deprecated
$socket = new React\Socket\Server(0);
$socket = new React\Socket\Server('127.0.0.1:8000');
$socket = new React\Socket\Server('127.0.0.1:8000', null, $context);
$socket = new React\Socket\Server('127.0.0.1:8000', $loop, $context);

// new
$socket = new React\Socket\SocketServer('127.0.0.1:0');
$socket = new React\Socket\SocketServer('127.0.0.1:8000');
$socket = new React\Socket\SocketServer('127.0.0.1:8000', $context);
$socket = new React\Socket\SocketServer('127.0.0.1:8000', $context, $loop);
```

- Feature: Update `Connector` signature to take optional `$context` as first argument. (#264 by @clue)

The new signature has been added to match the new `SocketServer` and consistently move the now commonly unneeded loop argument to the last argument. The previous signature has been deprecated and should not be used anymore. In its most basic form, both signatures are compatible.

```
// deprecated
$connector = new React\Socket\Connector(null, $context);
$connector = new React\Socket\Connector($loop, $context);

// new
$connector = new React\Socket\Connector($context);
$connector = new React\Socket\Connector($context, $loop);
```

1.8.0 (2021-07-11)

A major new feature release, see [release announcement](#).

- Feature: Simplify usage by supporting new default loop. (#260 by @clue)

```
// old (still supported)
$socket = new React\Socket\Server('127.0.0.1:8080', $loop);
$connector = new React\Socket\Connector($loop);

// new (using default loop)
```

```
$socket = new React\Socket\Server('127.0.0.1:8080');  
$connector = new React\Socket\Connector();
```

1.7.0 (2021-06-25)

- Feature: Support falling back to multiple DNS servers from DNS config. (#257 by @clue)

If you're using the default `Connector`, it will now use all DNS servers configured on your system. If you have multiple DNS servers configured and connectivity to the primary DNS server is broken, it will now fall back to your other DNS servers, thus providing improved connectivity and redundancy for broken DNS configurations.

- Feature: Use round robin for happy eyeballs DNS responses (load balancing). (#247 by @clue)

If you're using the default `Connector`, it will now randomize the order of the IP addresses resolved via DNS when connecting. This allows the load to be distributed more evenly across all returned IP addresses. This can be used as a very basic DNS load balancing mechanism.

- Internal improvement to avoid unhandled rejection for future Promise API. (#258 by @clue)
- Improve test suite, use GitHub actions for continuous integration (CI). (#254 by @SimonFrings)

1.6.0 (2020-08-28)

- Feature: Support upcoming PHP 8 release. (#246 by @clue)
- Feature: Change default socket backlog size to 511. (#242 by @clue)
- Fix: Fix closing connection when cancelling during TLS handshake. (#241 by @clue)
- Fix: Fix blocking during possible `accept()` race condition when multiple socket servers listen on same socket address. (#244 by @clue)
- Improve test suite, update PHPUnit config and add full core team to the license. (#243 by @SimonFrings and #245 by @WyriHaximus)

1.5.0 (2020-07-01)

- Feature / Fix: Improve error handling and reporting for happy eyeballs and immediately try next connection when one connection attempt fails. (#230, #231, #232 and #233 by @clue)

Error messages for failed connection attempts now include more details to ease debugging. Additionally, the happy eyeballs algorithm has been im-

proved to avoid having to wait for some timers to expire which significantly improves connection setup times (in particular when IPv6 isn't available).

- Improve test suite, minor code cleanup and improve code coverage to 100%. Update to PHPUnit 9 and skip legacy TLS 1.0 / TLS 1.1 tests if disabled by system. Run tests on Windows and simplify Travis CI test matrix for Mac OS X setup and skip all TLS tests on legacy HHVM. (#229, #235, #236 and #238 by @clue and #239 by @SimonFrings)

1.4.0 (2020-03-12)

A major new feature release, see **release announcement**.

- Feature: Add IPv6 support to **Connector** (implement “Happy Eyeballs” algorithm to support IPv6 probing). IPv6 support is turned on by default, use new **happy_eyeballs** option in **Connector** to toggle behavior. (#196, #224 and #225 by @WyriHaximus and @clue)
- Feature: Default to using DNS cache (with max 256 entries) for **Connector**. (#226 by @clue)
- Add **.gitattributes** to exclude dev files from exports and some minor code style fixes. (#219 by @reedy and #218 by @mmoreram)
- Improve test suite to fix failing test cases when using new DNS component, significantly improve test performance by awaiting events instead of sleeping, exclude TLS 1.3 test on PHP 7.3, run tests on PHP 7.4 and simplify test matrix. (#208, #209, #210, #217 and #223 by @clue)

1.3.0 (2019-07-10)

- Feature: Forward compatibility with upcoming stable DNS component. (#206 by @clue)

1.2.1 (2019-06-03)

- Avoid unneeded fragmented TLS work around for PHP 7.3.3+ and work around failing test case detecting EOF on TLS 1.3 socket streams. (#201 and #202 by @clue)
- Improve TLS certificate/passphrase example. (#190 by @jsor)

1.2.0 (2019-01-07)

- Feature / Fix: Improve TLS 1.3 support. (#186 by @clue)

TLS 1.3 is now an official standard as of August 2018! :tada: The protocol has major improvements in the areas of security, performance, and privacy. TLS 1.3 is supported by default as of OpenSSL 1.1.1. For example, this

version ships with Ubuntu 18.10 (and newer) by default, meaning that recent installations support TLS 1.3 out of the box :shipit:

- Fix: Avoid possibility of missing remote address when TLS handshake fails. (#188 by @clue)
- Improve performance by prefixing all global functions calls with \ to skip the look up and resolve process and go straight to the global function. (#183 by @WyriHaximus)
- Update documentation to use full class names with namespaces. (#187 by @clue)
- Improve test suite to avoid some possible race conditions, test against PHP 7.3 on Travis and use dedicated `assertInstanceOf()` assertions. (#185 by @clue, #178 by @WyriHaximus and #181 by @carusogabriel)

1.1.0 (2018-10-01)

- Feature: Improve error reporting for failed connection attempts and improve cancellation forwarding during DNS lookup, TCP/IP connection or TLS handshake. (#168, #169, #170, #171, #176 and #177 by @clue)

All error messages now always contain a reference to the remote URI to give more details which connection actually failed and the reason for this error. Accordingly, failures during DNS lookup will now mention both the remote URI as well as the DNS error reason. TCP/IP connection issues and errors during a secure TLS handshake will both mention the remote URI as well as the underlying socket error. Similarly, lost/dropped connections during a TLS handshake will now report a lost connection instead of an empty error reason.

For most common use cases this means that simply reporting the `Exception` message should give the most relevant details for any connection issues:

```
$promise = $connector->connect('tls://example.com:443');
$promise->then(function (ConnectionInterface $conn) use ($loop) {
    // ...
}, function (Exception $e) {
    echo $e->getMessage();
});
```

1.0.0 (2018-07-11)

- First stable LTS release, now following SemVer. We'd like to emphasize that this component is production ready and battle-tested. We plan to support all long-term support (LTS) releases for at least 24 months, so you have a rock-solid foundation to build on top of.

Contains no other changes, so it's actually fully compatible with the v0.8.12 release.

0.8.12 (2018-06-11)

- Feature: Improve memory consumption for failed and cancelled connection attempts. (#161 by @clue)
- Improve test suite to fix Travis config to test against legacy PHP 5.3 again. (#162 by @clue)

0.8.11 (2018-04-24)

- Feature: Improve memory consumption for cancelled connection attempts and simplify skipping DNS lookup when connecting to IP addresses. (#159 and #160 by @clue)

0.8.10 (2018-02-28)

- Feature: Update DNS dependency to support loading system default DNS nameserver config on all supported platforms (`/etc/resolv.conf` on Unix/Linux/Mac/Docker/WSL and WMIC on Windows) (#152 by @clue)

This means that connecting to hosts that are managed by a local DNS server, such as a corporate DNS server or when using Docker containers, will now work as expected across all platforms with no changes required:

```
$connector = new Connector($loop);
$connector->connect('intranet.example:80')->then(function ($connection) {
    // ...
});
```

0.8.9 (2018-01-18)

- Feature: Support explicitly choosing TLS version to negotiate with remote side by respecting `crypto_method` context parameter for all classes. (#149 by @clue)

By default, all connector and server classes support TLSv1.0+ and exclude support for legacy SSLv2/SSLv3. As of PHP 5.6+ you can also explicitly choose the TLS version you want to negotiate with the remote side:

```
// new: now supports 'crypto_method' context parameter for all classes
$connector = new Connector($loop, array(
    'tls' => array(
        'crypto_method' => STREAM_CRYPTO_METHOD_TLSv1_2_CLIENT
    )
));
```

- Minor internal clean up to unify class imports (#148 by @clue)

0.8.8 (2018-01-06)

- Improve test suite by adding test group to skip integration tests relying on internet connection and fix minor documentation typo. (#146 by @clue and #145 by @cn007b)

0.8.7 (2017-12-24)

- Fix: Fix closing socket resource before removing from loop (#141 by @clue)
This fixes the root cause of an uncaught `Exception` that only manifested itself after the recent `Stream v0.7.4` component update and only if you're using `ext-event` (`ExtEventLoop`).
- Improve test suite by testing against PHP 7.2 (#140 by @carusogabriel)

0.8.6 (2017-11-18)

- Feature: Add Unix domain socket (UDS) support to `Server` with `unix://` URI scheme and add advanced `UnixServer` class. (#120 by @andig)

// new: Server now supports "unix://" scheme
`$server = new Server('unix:///tmp/server.sock', $loop);`

// new: advanced usage
`$server = new UnixServer('/tmp/server.sock', $loop);`
- Restructure examples to ease getting started (#136 by @clue)
- Improve test suite by adding forward compatibility with PHPUnit 6 and ignore Mac OS X test failures for now until Travis tests work again (#133 by @gabriel-caruso and #134 by @clue)

0.8.5 (2017-10-23)

- Fix: Work around PHP bug with Unix domain socket (UDS) paths for Mac OS X (#123 by @andig)
- Fix: Fix `SecureServer` to return `null` URI if server socket is already closed (#129 by @clue)
- Improve test suite by adding forward compatibility with PHPUnit v5 and forward compatibility with upcoming `EventLoop` releases in tests and test Mac OS X on Travis (#122 by @andig and #125, #127 and #130 by @clue)
- Readme improvements (#118 by @jsor)

0.8.4 (2017-09-16)

- Feature: Add `FixedUriConnector` decorator to use fixed, preconfigured URI instead (#117 by @clue)

This can be useful for consumers that do not support certain URIs, such as when you want to explicitly connect to a Unix domain socket (UDS) path instead of connecting to a default address assumed by an higher-level API:

```
$connector = new FixedUriConnector(  
    'unix:///var/run/docker.sock',  
    new UnixConnector($loop)  
);  
  
// destination will be ignored, actually connects to Unix domain socket  
$promise = $connector->connect('localhost:80');
```

0.8.3 (2017-09-08)

- Feature: Reduce memory consumption for failed connections (#113 by @valga)
- Fix: Work around write chunk size for TLS streams for PHP < 7.1.14 (#114 by @clue)

0.8.2 (2017-08-25)

- Feature: Update DNS dependency to support hosts file on all platforms (#112 by @clue)

This means that connecting to hosts such as `localhost` will now work as expected across all platforms with no changes required:

```
$connector = new Connector($loop);  
$connector->connect('localhost:8080')->then(function ($connection) {  
    // ...  
});
```

0.8.1 (2017-08-15)

- Feature: Forward compatibility with upcoming EventLoop v1.0 and v0.5 and target evenement 3.0 a long side 2.0 and 1.0 (#104 by @clue and #111 by @WyriHaximus)
- Improve test suite by locking Travis distro so new defaults will not break the build and fix HHVM build for now again and ignore future HHVM build errors (#109 and #110 by @clue)
- Minor documentation fixes (#103 by @christiaan and #108 by @hansott)

0.8.0 (2017-05-09)

- Feature: New **Server** class now acts as a facade for existing server classes and renamed old **Server** to **TcpServer** for advanced usage. (#96 and #97 by @clue)

The **Server** class is now the main class in this package that implements the **ServerInterface** and allows you to accept incoming streaming connections, such as plaintext TCP/IP or secure TLS connection streams.

This is not a BC break and consumer code does not have to be updated.

- Feature / BC break: All addresses are now URIs that include the URI scheme (#98 by @clue)

```
- $parts = parse_url('tcp://' . $conn->getRemoteAddress());  
+ $parts = parse_url($conn->getRemoteAddress());
```
- Fix: Fix `unix://` addresses for Unix domain socket (UDS) paths (#100 by @clue)
- Feature: Forward compatibility with Stream v1.0 and v0.7 (#99 by @clue)

0.7.2 (2017-04-24)

- Fix: Work around latest PHP 7.0.18 and 7.1.4 no longer accepting full URIs (#94 by @clue)

0.7.1 (2017-04-10)

- Fix: Ignore HHVM errors when closing connection that is already closing (#91 by @clue)

0.7.0 (2017-04-10)

- Feature: Merge **SocketClient** component into this component (#87 by @clue)

This means that this package now provides async, streaming plaintext TCP/IP and secure TLS socket server and client connections for ReactPHP.

```
$connector = new React\Socket\Connector($loop);  
$connector->connect('google.com:80')->then(function (ConnectionInterface $conn) {  
    $connection->write('...');  
});
```

Accordingly, the **ConnectionInterface** is now used to represent both incoming server side connections as well as outgoing client side connections.

If you've previously used the **SocketClient** component to establish outgoing client connections, upgrading should take no longer than a few minutes.

All classes have been merged as-is from the latest v0.7.0 release with no other changes, so you can simply update your code to use the updated namespace like this:

```
// old from SocketClient component and namespace
$connector = new React\SocketClient\Connector($loop);
$connector->connect('google.com:80')->then(function (ConnectionInterface $conn) {
    $connection->write('...');
});

// new
$connector = new React\Socket\Connector($loop);
$connector->connect('google.com:80')->then(function (ConnectionInterface $conn) {
    $connection->write('...');
});
```

0.6.0 (2017-04-04)

- Feature: Add `LimitingServer` to limit and keep track of open connections (#86 by @clue)

```
$server = new Server(0, $loop);
$server = new LimitingServer($server, 100);

$server->on('connection', function (ConnectionInterface $connection) {
    $connection->write('hello there!' . PHP_EOL);
    ...
});
```

- Feature / BC break: Add `pause()` and `resume()` methods to limit active connections (#84 by @clue)

```
$server = new Server(0, $loop);
$server->pause();

$loop->addTimer(1.0, function() use ($server) {
    $server->resume();
});
```

0.5.1 (2017-03-09)

- Feature: Forward compatibility with Stream v0.5 and upcoming v0.6 (#79 by @clue)

0.5.0 (2017-02-14)

- Feature / BC break: Replace `listen()` call with URIs passed to constructor and reject listening on hostnames with `InvalidArgumentException` and

replace `ConnectionException` with `RuntimeException` for consistency (#61, #66 and #72 by @clue)

```
// old
$server = new Server($loop);
$server->listen(8080);

// new
$server = new Server(8080, $loop);
```

Similarly, you can now pass a full listening URI to the constructor to change the listening host:

```
// old
$server = new Server($loop);
$server->listen(8080, '127.0.0.1');

// new
$server = new Server('127.0.0.1:8080', $loop);
```

Trying to start listening on (DNS) host names will now throw an `InvalidArgumentException`, use IP addresses instead:

```
// old
$server = new Server($loop);
$server->listen(8080, 'localhost');

// new
$server = new Server('127.0.0.1:8080', $loop);
```

If trying to listen fails (such as if port is already in use or port below 1024 may require root access etc.), it will now throw a `RuntimeException`, the `ConnectionException` class has been removed:

```
// old: throws React\Socket\ConnectionException
$server = new Server($loop);
$server->listen(80);

// new: throws RuntimeException
$server = new Server(80, $loop);
```

- Feature / BC break: Rename `shutdown()` to `close()` for consistency throughout React (#62 by @clue)

```
// old
$server->shutdown();

// new
$server->close();
```

- Feature / BC break: Replace `getPort()` with `getAddress()` (#67 by @clue)

```
// old
echo $server->getPort(); // 8080

// new
echo $server->getAddress(); // 127.0.0.1:8080
```

- Feature / BC break: `getRemoteAddress()` returns full address instead of only IP (#65 by @clue)

```
// old
echo $connection->getRemoteAddress(); // 192.168.0.1

// new
echo $connection->getRemoteAddress(); // 192.168.0.1:51743
```

- Feature / BC break: Add `getLocalAddress()` method (#68 by @clue)

```
echo $connection->getLocalAddress(); // 127.0.0.1:8080
```

- BC break: The `Server` and `SecureServer` class are now marked `final` and you can no longer `extend` them (which was never documented or recommended anyway). Public properties and event handlers are now internal only. Please use composition instead of extension. (#71, #70 and #69 by @clue)

0.4.6 (2017-01-26)

- Feature: Support socket context options passed to `Server` (#64 by @clue)
- Fix: Properly return `null` for unknown addresses (#63 by @clue)
- Improve documentation for `ServerInterface` and lock test suite requirements (#60 by @clue, #57 by @shaunbramley)

0.4.5 (2017-01-08)

- Feature: Add `SecureServer` for secure TLS connections (#55 by @clue)
- Add functional integration tests (#54 by @clue)

0.4.4 (2016-12-19)

- Feature / Fix: `ConnectionInterface` should extend `DuplexStreamInterface` + documentation (#50 by @clue)
- Feature / Fix: Improve test suite and switch to normal stream handler (#51 by @clue)

- Feature: Add examples (#49 by @clue)

0.4.3 (2016-03-01)

- Bug fix: Suppress errors on stream_socket_accept to prevent PHP from crashing
- Support for PHP7 and HHVM
- Support PHP 5.3 again

0.4.2 (2014-05-25)

- Verify stream is a valid resource in Connection

0.4.1 (2014-04-13)

- Bug fix: Check read buffer for data before shutdown signal and end emit (@ArtyDev)
- Bug fix: v0.3.4 changes merged for v0.4.1

0.3.4 (2014-03-30)

- Bug fix: Reset socket to non-blocking after shutting down (PHP bug)

0.4.0 (2014-02-02)

- BC break: Bump minimum PHP version to PHP 5.4, remove 5.3 specific hacks
- BC break: Update to React/Promise 2.0
- BC break: Update to Evenement 2.0
- Dependency: Autoloading and filesystem structure now PSR-4 instead of PSR-0
- Bump React dependencies to v0.4

0.3.3 (2013-07-08)

- Version bump

0.3.2 (2013-05-10)

- Version bump

0.3.1 (2013-04-21)

- Feature: Support binding to IPv6 addresses (@clue)

0.3.0 (2013-04-14)

- Bump React dependencies to v0.3

0.2.6 (2012-12-26)

- Version bump

0.2.3 (2012-11-14)

- Version bump

0.2.0 (2012-09-10)

- Bump React dependencies to v0.2

0.1.1 (2012-07-12)

- Version bump

0.1.0 (2012-07-11)

- First tagged release